Լաբորատոր աշխատանք 11

Օբյեկտի փոխանցումը ֆունկցիային

Աշխատանքի նպատակը: Տարբերակել օբյեկտի փոխանցումը արժեքով, հղումով և հաստատուն հղումով:

Խնդրի պահանջը:

Բանկային հաշվի գործարքներ Տվյալներ:

Ստեղծել AccountLedger դաս՝ պահելով գործարքների գումարների զանգվածը և p սկզբնական մսացորդը: Դրականը մուտք է, բացասականը՝ ելք: Հաշվարկներ: Հաշվել վերջնական մսացորդը, գտնել գործարքից հետո ստացված ամենափոքր մսացորդի ինդեքսը և այն գործարքների քանակը, որոնցից հետո մսացորդը բացասական է: Թույլատրելի արժեքներ: p≥0, գործարքները վերջավոր են: Ուսումնական մոդելում բացասական մսացորդը թույլատրվում է: Ֆունկցիաներ: analyze(const AccountLedger&) արտածում է երեք հաշվարկները: changeCopy(AccountLedger object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const AccountLedger&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո: Փորձնական տվյալներ: transactions={5000,-12000,3000,-2000}, p=6000: Ինդեքսները սկսվում են 0-ից:

Տեսական մաս: Արժեքով փոխանցումը ստեղծում է պարամետրի պատճեն: Հղումով փոխանցումը թույլ է տալիս փոխել սկզբնական օբյեկտը: const հղումը կիրառվում է առանց պատճենման ընթերցելու համար: Արտաքին սովորական ֆունկցիան private տվյալները ստանում է public մեթոդների միջոցով:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <limits>
#include <stdexcept>
using namespace std;
class AccountLedger {
```

```
private:
    vector<double> transactions;
    double p;
public:
    AccountLedger(const vector<double>& t, double initial) {
        if (t.size() < 3 || t.size() > 30 ||
            !isfinite(initial) || initial < 0) {
            throw invalid_argument("Invalid account data");
        }
        for (double x : t) {
            if (!isfinite(x)) {
                throw invalid_argument("Invalid transaction");
            }
        }
        transactions = t;
        p = initial;
    }
    int getSize() const {
        return static_cast<int>(transactions.size());
    }
    double getP() const {
        return p;
    }
    double get(int index) const {
        return transactions.at(index);
    }
    bool set(int index, double value) {
```

```
    if (index < 0 || index >= getSize() || !isfinite(value)) {
        return false;
    }
    transactions[index] = value;
    return true;
}
double finalBalance() const {
    double balance = p;
    for (double x : transactions) {
        balance += x;
    }
    return balance;
}
int minBalanceIndex() const {
    double balance = p;
    double minBalance = numeric_limits<double>::infinity();
    int minIndex = -1;
    for (int i = 0; i < getSize(); i++) {
        balance += transactions[i];
        if (balance < minBalance - 1e-9) {
            minBalance = balance;
            minIndex = i;
        }
    }
    return minIndex;
}
int negativeCount() const {
```

```
        double balance = p;
        int count = 0;
        for (double x : transactions) {
            balance += x;
            if (balance < -1e-9) {
                count++;
            }
        }
        return count;
    }
    void print() const {
        cout << "Initial balance: " << p << endl;
        cout << "Transactions: ";
        for (double x : transactions) {
            cout << x << " ";
        }
        cout << endl;
    }
};
void analyze(const AccountLedger& object) {
    cout << "Final balance: "
         << object.finalBalance() << endl;
    cout << "Minimum balance index: "
         << object.minBalanceIndex() << endl;
    cout << "Negative balance count: "
         << object.negativeCount() << endl;
}
```

```
void changeCopy(AccountLedger object, int index, double value) {
    if (object.set(index, value)) {
        object.print();
    }
}
AccountLedger reversed(const AccountLedger& object) {
    vector<double> temp;
    for (int i = object.getSize() - 1; i >= 0; i--) {
        temp.push_back(object.get(i));
    }
    return AccountLedger(temp, object.getP());
}
int main() {
    int n;
    double p;
    cout << "Enter number of transactions (3-30): ";
    cin >> n;
    while (n < 3 || n > 30) {
        cout << "Invalid input! Try again: ";
        cin >> n;
    }
    cout << "Enter initial balance: ";
    cin >> p;
    while (!isfinite(p) || p < 0) {
        cout << "Invalid input! Try again: ";
        cin >> p;
    }
```

```
    vector<double> t(n);
    for (int i = 0; i < n; i++) {
        cout << "Enter transaction " << i << ": ";
        cin >> t[i];
        while (!isfinite(t[i])) {
            cout << "Invalid input! Try again: ";
            cin >> t[i];
        }
    AccountLedger account(t, p);
    account.print();
    analyze(account);
    account.print();
    changeCopy(account, 0, 7000)
    account.print();
    AccountLedger rev = reversed(account);
    rev.print();
    account.print()
    return 0;
}
```